GPU Prime Factorization with NVIDIA CGBN

Design and Implementation of `cgbnprimefactors.cu` and `ecmcgbnprimefactorsmpi.cu`

Stefan Teleman <stefan.teleman@gmail.com>

August 27, 2026

Abstract

This document describes, in detail, the design and implementation of two GPU prime-factorization programs. Both print the *distinct* prime factors (no multiplicity) of an unsigned integer and share a command-line convention and an output format. Both perform all multiple-precision integer arithmetic on the GPU through NVIDIA CGBN (installed at `/usr/local/include/cgbn`) and distribute work across ranks with OpenMPI (installed at `/usr/lib64/openmpi`). They differ in algorithm and in how they parallelize: `cgbnprimefactors.cu` splits composites with massively parallel Pollard's rho backed by GPU Miller–Rabin, distributing the rho search over MPI ranks; `ecmcgbnprimefactorsmpi.cu` trial-divides by a table of proven primes, distributed three ways (OpenMPI × pthreads × CUDA/CGBN), with GMP-ECM as the residual fallback. Neither is intended to beat the CPU reference on hard semiprimes; their value is the demonstration of CGBN big-integer parallelism and the distributed orchestration around it.

Contents

1 Shared foundations

1.1 Common contract

Both programs share the same external contract, which keeps them drop-in comparable against the CPU reference `primefactors.c`:

- **Output is the set of distinct prime factors**, de-duplicated (a prime power contributes its base once) and sorted ascending.
- **Output is written to `stderr`**, framed by rule lines, in the form `Prime Factors of <N>: <p1> <p2> ...`.
- **Only MPI rank 0 prints.** Every rank computes an identical factor list, so no gather of results for printing is needed (`cgbnprimefactors.cu`); or the results are gathered to rank 0 first (`ecmcgbnprimefactorsmpi.cu`).
- **`-np 1` (or no `mpirun`) is the serial case.** With `g_nranks == 1` each program degenerates exactly to its single-process algorithm; the distributed and serial paths are the same code.

1.2 NVIDIA CGBN

CGBN (Cooperative Groups Big Numbers) is a header-only library at `/usr/local/include/cgbn` that performs fixed-width multiple-precision integer arithmetic across many parallel *instances*, where each instance is a cooperating group of `TPI` (threads-per-instance) GPU threads. The bit width is a compile-time template parameter, and `TPI` must divide the warp and be at most 32.

The single include `<cgbn/cgbn.h>` selects a backend by preprocessor state: the CUDA device backend when `__CUDA_ARCH__` is defined, and the host/GMP backend (`cgbn_mpz.h`) when `__GMP_H__` is defined — hence `<gmp.h>` is included *before* `<cgbn/cgbn.h>` in both programs so the host compile has a valid backend (otherwise CGBN errors with "You must use GMP for now"). At the tail of `cgbn.h`, the signed two's-complement wrappers in `cgbn_signed.h` are auto-included; that header has *no include guard*, so it must never be included a second time by the program (a double include is a redefinition error).

The device primitives used across the two programs are: `cgbn_load`/`cgbn_store` (device memory ↔ register `cgbn_t`), `cgbn_sqr_wide` + `cgbn_rem_wide` (a full double-width square followed by reduction — the exactness lynchpin of the modular steps), `cgbn_add_ui32`/`cgbn_sub`/`cgbn_compare`, `cgbn_gcd`, `cgbn_modular_power`, `cgbn_div_rem`, and the signed `cgbn_signed_rem`.

1.3 OpenMPI

OpenMPI (5.0.5) lives at `/usr/lib64/openmpi`. The Makefile derives the compiler/linker flags from the `mpicxx` wrapper via `-showme` rather than hard-coding paths (§4). Both binaries are ordinary MPI programs launched with `mpirun`. The MPI runtime is not on the default `PATH` on this host, so `mpirun` needs `PATH=/usr/lib64/openmpi/bin:$PATH` to find its `prterun` launcher; the library path is embedded as `RUNPATH` in the binaries, so `LD_LIBRARY_PATH` is not required at run time.

1.4 Host bignum

Both programs carry a small host-side big-integer type used only for decimal I/O, small-value arithmetic, and marshalling to/from device memory — never for the hot arithmetic, which is on the GPU:

```
1 #define MAXBITS 32768
2 #define LIMBS   (MAXBITS / 32)
```

```
struct BN { uint32_t l[LIMBS]; };
```

Listing 1: The host bignum: fixed-width little-endian 32-bit limbs.

`MAXBITS = 32768` matches the widest CGBN engine, so any value the GPU can process fits a `BN`. Helpers provide comparison (`bn_cmp`), bit length via `__builtin_clz` (`bn_bitlen`), short division/-modulo by a 32-bit divisor (`bn_divmod_u32`, `bn_mod_u32`), and decimal conversion (`bn_from_dec`, `bn_to_dec`). Marshalling to a width-specific device buffer is templated:

```
template <uint32_t BITS>
static void bn_to_mem(const BN& v, cgbn_mem_t<BITS>& m) {
  const int LB = BITS / 32;
  for (int i = 0; i < LB; ++i) m._limbs[i] = (i < LIMBS) ? v.l[i] : 0;
}
```

Listing 2: Copying a `BN` into a width-`BITS` CGBN memory image (upper limbs zeroed).

1.5 Compile-time width table and runtime dispatch

Both programs template their kernels on `<BITS, TPI>` and instantiate them at nine widths, driven by a single X-macro so that adding or removing a width is a one-line edit:

```
#define WIDTH_LIST(X)   \
  X(128,   4,  E128)  X(256,  8, E256)  X(512,  16, E512)  \
  X(1024,  32, E1024) X(2048, 32, E2048) X(4096, 32, E4096) \
  X(8192,  32, E8192) X(16384,32, E16384) X(32768,32, E32768)
```

Listing 3: The `WIDTH_LIST` X-macro (from `cgbnprimefactors.cu`).

`TPI` packs one 32-bit limb per thread up to 1024 bits (128/4, 256/8, 512/16, 1024/32); beyond that `TPI` holds at CGBN's maximum of 32 and each thread carries 2–32 limbs. The X-macro is expanded three ways: to *declare* the engines, to *initialize* them, and to build the *dispatch* ladder that picks a width at run time. The dispatch rule is uniform: choose the *narrowest* width whose bit budget covers the operand with 16 bits of headroom (`BITS - 16`). The narrow-width preference is a performance decision (fewer limbs ⇒ fewer ALU ops); the 16-bit headroom is a correctness one (it keeps a small additive constant, or a clear sign bit, from overflowing the width). Both programs cap the input at `MAXBITS - 16` bits up front, so the widest engine always matches.

2 `cgbnprimefactors.cu`: parallel Pollard's rho

2.1 Strategy overview

`cgbnprimefactors.cu` factors N with a three-stage pipeline in which the host orchestrates and the GPU does all multiple-precision arithmetic:

1. **Host trial division** peels off small prime factors ($<$ `SMALL_BOUND` $= 100000$) using the host `BN` type (no GMP). Every surviving cofactor is odd and has all prime factors above the bound, hence larger than every Miller–Rabin base used later.
2. **GPU Miller–Rabin** (`mr_kernel`) decides whether a cofactor is prime, one instance per witness base.
3. **GPU Pollard's rho** (`rho_kernel`) launches thousands of instances, each walking $f(x) = x^2 + c \bmod N$ with a distinct constant c; the first to expose a nontrivial `gcd` claims a shared slot and stores the factor.

A work stack drives the recursion: each split pushes the divisor and the exact quotient (from `divexact_kernel`) back onto the stack until every entry is prime. The distinct-factor set is sorted before printing.

2.2 The rho step and the correctness lynchpin

The core iteration is one Pollard's-rho step $r \leftarrow (x^2 + c) \bmod M$:

```
template <uint32_t BITS, uint32_t TPI>
__device__ __forceinline__ void
rho_step(cgbn_env_t<cgbn_context_t<TPI>, BITS> env, cgbn_t& r,
         const cgbn_t& x, uint32_t c, const cgbn_t& M) {
  cgbn_wide_t w;
  cgbn_sqr_wide(env, w, x);     // full 2*BITS-bit square (no truncation)
  cgbn_rem_wide(env, r, w, M);  // reduce the wide product modulo M
  cgbn_add_ui32(env, r, r, c);
  if (cgbn_compare(env, r, M) >= 0) cgbn_sub(env, r, r, M);
}
```

Listing 4: `rho_step`: one $x \mapsto x^2 + c \bmod M$ evaluation.

The square is computed at *double* width (`cgbn_sqr_wide` → `cgbn_wide_t`) and only then reduced (`cgbn_rem_wide`). A plain same-width `cgbn_mul` would truncate x^2 to `BITS` bits and corrupt the modular reduction; the wide-square-then-reduce pair is what makes the arithmetic exact. The final conditional subtract normalizes the $+c$ addition back into $[0, M)$; because $c < 2^{19}$ and the width carries ≥ 16 bits of headroom, this single subtract always suffices.

2.3 The rho kernel

Each instance walks with its own constant and races the others:

```
int instance = (blockIdx.x * blockDim.x + threadIdx.x) / TPI;
if (instance >= (int) count) return;
context_t ctx(cgbn_report_monitor, report, instance);
env_t env = ctx.template env<env_t>();

cgbn_t M, x, y, diff, g;
cgbn_load(env, M, g_M);
cgbn_set_ui32(env, x, 2);
cgbn_set_ui32(env, y, 2);
uint32_t c = c_base + (uint32_t) instance + 1;    // this instance's constant

for (uint32_t i = 0; i < max_iters; ++i) {
  if ((i & 4095) == 0 && *(volatile int*) g_found) return;  // early out
  rho_step(env, x, x, c, M);          // tortoise: one step
  rho_step(env, y, y, c, M);          // hare: two steps
  rho_step(env, y, y, c, M);
  int cmp = cgbn_compare(env, x, y);
  if (cmp == 0) break;                // cycle closed with no factor
  cgbn_sub(env, diff, /* larger */, /* smaller */);
  cgbn_gcd(env, g, diff, M);
  if (cgbn_equals_ui32(env, g, 1)) continue;
  if (cgbn_compare(env, g, M) == 0) break;
  // nontrivial gcd: claim the shared slot on lane 0, broadcast, store
  int win = 0;
  if ((threadIdx.x % TPI) == 0) win = (atomicCAS(g_found, 0, 1) == 0);
  win = __shfl_sync(0xffffffff, win, (threadIdx.x / TPI) * TPI);
  if (win) cgbn_store(env, g_factor, g);
  return;
}
```

Listing 5: `rho_kernel`: Floyd tortoise/hare, one distinct c per instance.

Salient points:

- **Floyd cycle detection.** The tortoise `x` advances one step, the hare `y` two; a nontrivial $\gcd(|x-y|, M)$ is a factor. Each step's `gcd` is independent, which exposes instruction-level parallelism (see §2.8).
- **Independent walks.** Instance i uses $c = c_{\text{base}} + i + 1$, so the instances collectively cover a contiguous band of constants; the band is chosen by the caller (§2.6).
- **Cooperative claim.** A finder does an `atomicCAS` on lane 0 of its instance, `__shfl_sync`-broadcasts the win across the instance's `TPI` lanes, and only then all lanes cooperatively `cgbn_store` the factor (a store is a group operation, so every lane must agree to participate).
- **Bounded polling early-out.** Every 4096 iterations the instance reads the shared `g_found` flag through a `volatile` pointer and bails if another instance already won, so a decided launch drains quickly.

2.4 Miller–Rabin and exact-division kernels

`mr_kernel` runs one instance per witness base (`MR_BASES = {2,3,5,...,37}`). It factors $M-1 = d \cdot 2^s$ via `cgbn_shift_right`, computes $a^d \bmod M$ with `cgbn_modular_power`, and runs the squaring chain; a base that witnesses compositeness sets a shared flag with `atomicOr`. As in rho, the squaring step uses `cgbn_sqr_wide` + `cgbn_rem_wide`. `M` is guaranteed odd and larger than every base by the host trial-division stage. `divexact_kernel` is a single-instance $q = M/d$ via `cgbn_div_rem` (the remainder is known to be zero: `d` came from a rho split).

2.5 The per-width engine

Each width is wrapped in an `Engine<BITS,TPI>` that owns its device buffers and CGBN error report and exposes `is_prime`, `rho_once`, and `divexact`. Instances are scaled so that the *warp count* is roughly constant across widths:

```
static constexpr uint32_t INSTANCES = RHO_WARPS * 32U / TPI;  // RHO_WARPS = 8192
```

Listing 6: Instance scaling keeps ∼8192 warps live at every width.

A narrow width has a small `TPI`, so it packs more instances per warp; to keep every SM busy it therefore needs *more* instances. Fixing the warp count rather than the instance count is what keeps occupancy high after the width-dispatch change (§2.8). `rho_once` runs a single attempt over the constant band $c \in [\,\text{slot} \cdot \text{INSTANCES} + 1,\ (\text{slot}+1) \cdot \text{INSTANCES}\,]$ and returns whether this GPU split M:

```
bool rho_once(const BN& M, BN& factor, uint32_t slot, uint32_t iters) {
  put(d_M, M);                              // H2D of the modulus
  /* clear d_flag */
  uint32_t threads = INSTANCES * TPI;
  uint32_t blocks  = (threads + RHO_BLOCK - 1) / RHO_BLOCK;
  uint32_t c_base  = slot * INSTANCES;      // disjoint band per (rank, attempt)
  rho_kernel<BITS,TPI><<<blocks,RHO_BLOCK>>>(report, d_M, INSTANCES, iters,
                                             c_base, d_flag, d_factor);
  cudaDeviceSynchronize(); CGBN_CHECK(report);
  /* read d_flag; if set, copy d_factor back into `factor` and return true */
}
```

Listing 7: `Engine::rho_once`: one attempt over a slot's disjoint constant band.

2.6 Distributed rho over MPI

Pollard's rho is embarrassingly parallel over the walk constant c, and c is the axis MPI distributes. The design is **SPMD with a replicated work stack**: every rank runs the *identical* orchestration loop on identical data. Trial division, Miller–Rabin, and exact division are deterministic and cheap, so each rank recomputes them locally and stays in lockstep with **zero communication**. `mpirun` already replicates `argv`, so all ranks parse the same N. The ranks diverge *only* inside the rho search:

```
static bool mpi_rho(const BN& C, BN& factor) {
  uint32_t iters = 200000U;
  for (int attempt = 0; attempt < 7; ++attempt) {
    uint32_t slot = (uint32_t) attempt * g_nranks + g_rank;  // disjoint per pair
    BN local; bn_zero(local);
    int found = gpu_rho_once(C, local, slot, iters) ? 1 : 0;

    int cand   = found ? g_rank : INT_MAX;   // INT_MAX == "found nothing"
    int winner = INT_MAX;
    MPI_Allreduce(&cand, &winner, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);
    if (winner != INT_MAX) {
      factor = local;                        // valid on the winner
      MPI_Bcast(&factor, sizeof(BN), MPI_BYTE, winner, MPI_COMM_WORLD);
      return true;                           // all ranks now hold the same factor
    }
    iters *= 3;                              // nobody split it: escalate together
  }
  return false;
}
```

Listing 8: `mpi_rho`: disjoint slices, elect the lowest finder, broadcast the split.

Because `slot = attempt·nranks + rank`, every (rank, attempt) pair searches a *disjoint* constant slice — no two GPUs ever repeat a walk. After each attempt the ranks synchronize: `MPI_Allreduce(MIN)` over `cand` (a finder contributes its rank, a non-finder `INT_MAX`) deterministically elects the lowest-ranked finder, and `MPI_Bcast` hands that rank's factor (a fixed-size `BN`, sent as `MPI_BYTE`) to everyone. All ranks return the same d, so their replicated stacks stay identical. If no rank split the cofactor, all escalate the walk length `iters *= 3` in lockstep and advance to the next band. With `g_nranks == 1` this is exactly the serial seven-attempt escalating search. `gpu_init` binds each rank to GPU `rank % deviceCount`, so one rank per distinct GPU is the intended deployment; ranks beyond the device count round-robin and share a device (correct, but contended).

2.7 Main control flow

`main` initializes MPI, parses N identically on every rank, caps it at `MAXBITS - 16` bits, and binds a GPU. Host trial division strips small primes into the working value `M`. Surviving `M` (if > 1) seeds the work stack; the loop pops a cofactor C, records it if `gpu_is_prime` says prime, else calls `mpi_rho` and pushes the divisor and `gpu_divexact` quotient. Finally the factor set is sorted and rank 0 prints. If both rho attempts and the escalation are exhausted without a split, the code emits a `warning:` and records the composite unfactored — a deliberate termination guard, not an expected outcome.

2.8 Performance reality and profiling

This program *demonstrates* massively parallel factorization; it does *not* beat the CPU. Parallel rho gains only $\sim\sqrt{k}$ from k walks and scales $O(\sqrt{\text{factor}})$, so it is throughput-oriented, not a hard-semiprime cracker. Profiling (nsys + ncu, RTX 4080) shows `rho_kernel` at 99.8% of GPU

time, *latency-bound on dependent integer-ALU chains* (DRAM ≈ 0%; top stall is fixed-latency execution dependency). Two structural wins are captured in the shipped code:

1. **Per-cofactor width dispatch.** Running a ∼100-bit cofactor through the 128-bit kernel instead of a fixed 512-bit engine cut per-iteration op-count ∼4×.
2. **Instance scaling.** Matching `TPI` to width initially starved the GPU; `INSTANCES = RHO_WARPS*32/TPI` restored ∼8192 warps and occupancy from 55% to 74%.

Together these took the 16-digit-factor benchmark from ∼37 s to ∼8.65 s (∼4.3×). Three further "optimizations" were implemented, profiled, and reverted as net-slower, and the reversions are documented in the code: *Brent's batched-gcd* (∼2.6× slower — `cgbn_gcd` is not the bottleneck and Brent's serial product chain worsens the latency stall), *per-instance ILP* (2–4× slower — walk-level parallelism is already saturated and packing walks only raises register pressure), and `__launch_bounds__` *register-capping* to force occupancy (monotonically slower — the compiler's default register/occupancy point is optimal). The lesson: the kernel is throughput-bound on CGBN integer arithmetic at the compiler's default allocation; the remaining wins are algorithmic, not tuning knobs.

3 `ecmcgbnprimefactorsmpi.cu`: distributed trial division + ECM

3.1 Strategy overview

`ecmcgbnprimefactorsmpi.cu` takes a completely different route: it trial-divides N by a precomputed table of *proven* primes read from a file, distributing the trial-division work three ways, with GMP-ECM as the fallback for whatever survives.

- **OpenMPI** partitions the divisor *value* interval $[1, \sqrt{N}]$ into `nranks` equal sub-intervals; each rank keeps only the file primes whose value lands in its sub-range.
- **pthreads** split each rank's prime slice into contiguous index chunks — one host thread per chunk, each driving its own CUDA stream and device buffers.
- **CUDA/CGBN** does the arithmetic: `mod_kernel` runs one CGBN instance per prime computing $N \bmod p$ and flagging $p \mid N$.

The command line is

```
mpirun -np <ranks> ./ecmcgbnprimefactorsmpi <primes-file> <N> [threads]
```

where `[threads]` defaults to the number of online CPUs.

3.2 The proven-primes assumption

The central assumption is that every number in the file has been *deterministically* proven prime (not merely a probabilistic Miller–Rabin pass). This lets the program record a divisor hit as a prime factor *directly*, with no primality test on the divisor. Miller–Rabin is run in exactly one place — on the ECM *residual* cofactor (§3.6) — and that residual is never a file prime, so the assumption is never violated.

3.3 The trial-division kernel and signed remainder

One CGBN instance per prime computes $N \bmod p$:

```
int instance = (blockIdx.x * blockDim.x + threadIdx.x) / TPI;
if (instance >= (int) count) return;
context_t ctx(cgbn_report_monitor, report, instance);
env_t env = ctx.template env<env_t>();

cgbn_t N, p, r;
cgbn_load(env, N, g_N);
cgbn_load(env, p, &g_primes[instance]);
```

```
cgbn_signed_rem(env, r, N, p);              // signed two's-complement remainder
uint32_t isdiv = cgbn_equals_ui32(env, r, 0) ? 1u : 0u;
if ((threadIdx.x % TPI) == 0) g_div[instance] = isdiv;   // lane 0 writes
```

Listing 9: `mod_kernel`: N mod p per instance via CGBN's *signed* remainder.

The remainder deliberately uses `cgbn_signed_rem` from CGBN's signed two's-complement support (in `cgbn_signed.h`, auto-included by `cgbn.h`; the program must *not* include it a second time). N and p are positive and the width carries ≥ 16 bits of headroom, so the sign bit is clear and the signed remainder equals the true unsigned remainder — using the signed API exercises that support directly on values where its result is provably identical to the unsigned one. Because all remainders share the same N, the CGBN width is chosen *once* from bitlen(N), not per prime (`trial_divide_dispatch`, using the same `WIDTH_LIST`/`BITS-16` scheme as §1.5).

3.4 The pthread worker

Each host thread owns a private CUDA stream, a private CGBN error report, and private device buffers, and processes its index slice in `MOD_BATCH` (2^{16}) sized launches so device memory stays bounded regardless of table size:

```
const uint32_t cap = (total < MOD_BATCH) ? (uint32_t) total : MOD_BATCH;
cudaStream_t stream; cudaStreamCreate(&stream);
/* d_N, d_primes[cap], d_div[cap] via cudaMalloc; H2D of N on `stream` */
for (size_t off = 0; off < total; off += cap) {
  uint32_t n = min(cap, total - off);
  /* pack n prime images into pinned-order host buffer, async H2D on stream */
  mod_kernel<BITS,TPI><<<blocks, MOD_BLOCK, 0, stream>>>(report, d_N,
                                                         d_primes, n, d_div);
  /* async D2H of the divisibility flags; cudaStreamSynchronize(stream) */
  for (uint32_t i = 0; i < n; ++i)
    if (hdiv[i]) a->out->push_back(a->primes[a->start + off + i]);
}
```

Listing 10: `worker`: per-thread stream, bounded batches, async H2D/compute/D2H.

Each thread accumulates its hits into a thread-local vector; there is no shared mutable state between threads during the scan, so no locking is needed. The CUDA runtime is thread-safe, and the per-thread streams let one rank overlap H2D copies on one chunk with compute on another. `trial_divide` spawns the threads over contiguous `chunk = ceil(np / nthreads)` index ranges with `pthread_create`, then `pthread_joins` and concatenates the per-thread hit lists.

3.5 MPI interval partition

Every rank reads the whole file and parses `argv` itself (SPMD, no broadcast of input). $\sqrt{N}$ is computed once with `mpz_sqrt`. The interval $[1, \sqrt{N}]$ is tiled into half-open value sub-intervals [lo, hi):

```
mpz_fdiv_q_ui(width, sqrtNz, g_nranks);       // width = floor(sqrt(N) / ranks)
mpz_mul_ui(lo, width, g_rank); mpz_add(lo, lo, one);          // lo = rank*width + 1
if (g_rank == g_nranks - 1) mpz_add(hi, sqrtNz, one);         // last rank: to sqrt(N)
else { mpz_mul_ui(hi, width, g_rank + 1); mpz_add(hi, hi, one); }
```

Listing 11: .]Each rank's half-open value sub-interval of [1, sqrt(N)].

The last rank absorbs the rounding remainder so the ranks tile $[1, \sqrt{N}]$ exactly, and each prime falls in exactly one rank's interval — so there are no cross-rank duplicates and no communication is needed until the final gather. While reading the file each rank keeps a token only if it is ≥ 2, $\leq \sqrt{N}$, and inside [lo, hi).

3.6 Gather and the ECM residual

After the local scan, `MPI_Gather` collects the per-rank hit counts and `MPI_Gatherv` collects the found primes (as `MPI_BYTE` blobs of `BN`) to rank 0. Rank 0 records each found prime, divides its full multiplicity out of a working copy of N, and hands any residual cofactor > 1 to `factor_residual`. A residual arises when a prime factor exceeds $\sqrt{N}$ (so it was never in the divisor interval) or the table was incomplete:

```
static void factor_residual(const mpz_t n) {
  if (mpz_cmp_ui(n, 1UL) == 0) return;
  if (mpz_probab_prime_p(n, MR_ROUNDS) > 0) {   // residual only; never a file prime
    record_factor_mpz(n); return;
  }
  mpz_t d, q; mpz_init(d); mpz_init(q);
  if (ecm_find_factor(d, n)) {
    mpz_divexact(q, n, d);
    factor_residual(d); factor_residual(q);     // recurse on both pieces
  } else {
    /* warning + record the composite unfactored (termination guard) */
  }
  mpz_clear(d); mpz_clear(q);
}
```

Listing 12: `factor_residual`: Miller–Rabin gate, then recursive ECM splitting.

`ecm_find_factor` is GMP-ECM (`libecm`), single-threaded because the residual is a rare, small tail of the total work. It walks an `ECM_LADDER` of `{B1, curves}` rows following GMP-ECM's tables (from `{2000, 30}` up to `{2.9e9, 520000}`), targeting factor sizes from ∼15 to ∼70 digits; each curve resets `B1done`, `x`, and `sigma` to force a fresh curve at the level's bound, with a fixed RNG seed for reproducibility. The first curve to return a proper factor $1 < d < n$ wins.

3.7 Main control flow

`main` initializes MPI; validates the argument count; parses N and caps it at `MAXBITS - 16` bits; computes $\sqrt{N}$; binds GPU `rank % deviceCount`; derives this rank's [lo, hi); reads and filters the prime file; runs `trial_divide_dispatch` (pthreads × CUDA); gathers hits to rank 0; and on rank 0 records factors, divides out multiplicities, runs `factor_residual` on any cofactor, sorts, and prints. With one rank it scans the whole $[1, \sqrt{N}]$ interval — identical to serial. Edge inputs $N \in \{0, 1\}$ produce an empty factor set.

4 Build system and toolchain

Neither GPU target is part of `make all` (which builds only the CUDA-less CPU `primefactors`), so a host without CUDA can still build the reference. The relevant Makefile machinery:

```
MPICXX       = /usr/lib64/openmpi/bin/mpicxx
MPI_LIBDIRS  = $(shell $(MPICXX) --showme:libdirs)
MPI_INCFLAGS = $(addprefix -I,$(shell $(MPICXX) --showme:incdirs))
MPI_LIBFLAGS = $(addprefix -L,$(MPI_LIBDIRS)) \
               $(addprefix -l,$(shell $(MPICXX) --showme:libs))
# Emit DT_RUNPATH (modern tag) so the binaries find libmpi without LD_LIBRARY_PATH
MPI_RPATHFLAGS = -Xlinker --enable-new-dtags \
                 $(foreach d,$(MPI_LIBDIRS),-Xlinker -rpath -Xlinker $(d))

NVCCFLAGS = -O3 -arch=sm_89 -I/usr/local/include $(MPI_INCFLAGS)
NVCCLIBS  = -lgmp $(MPI_LIBFLAGS) $(MPI_RPATHFLAGS)

cgbnprimefactors: cgbnprimefactors.cu
```

```
14          $(NVCC) $(NVCCFLAGS) $< -o $@ $(NVCCLIBS)
15
16  ecmcgbnprimefactorsmpi: ecmcgbnprimefactorsmpi.cu
17          $(NVCC) $(NVCCFLAGS) $< -o $@ -lecm $(NVCCLIBS) -lpthread
```

Listing 13: OpenMPI flags derived from the `mpicxx` wrapper; libdir embedded as RUNPATH.

Key toolchain facts:

- **nvcc** (`/usr/local/cuda-12.9`), `-arch=sm_89` (RTX 4080). Change `GPU_ARCH` in the Makefile for another GPU.
- **CGBN** at `/usr/local/include/cgbn` (header-only; brought in by `-I/usr/local/include`). Its host backend requires GMP, which is why `<gmp.h>` precedes `<cgbn/cgbn.h>`.
- **OpenMPI** include/lib flags come from `mpicxx -showme`; nvcc consumes `-I/-L/-l` natively, so no `-ccbin mpicxx` swap is needed. The libdir is embedded as `DT_RUNPATH`.
- `ecmcgbnprimefactorsmpi` additionally links `-lecm` (before `-lgmp`: `libecm` depends on `libgmp`) and `-lpthread`.
- At run time `mpirun` must be on `PATH` (`export PATH=/usr/lib64/openmpi/bin:$PATH`); the library path is already in the binary's `RUNPATH`.

5 Comparison and closing notes

	`cgbnprimefactors.cu`	`ecmcgbnprimefactorsmpi.cu`
Algorithm	Pollard's rho + Miller–Rabin	trial division + ECM
Divisor source	searched (c-space walks)	proven-prime table (file)
Parallel axes	CUDA + OpenMPI	CUDA + pthreads + OpenMPI
MPI splits	the rho constant space c	the interval $[1, \sqrt{N}]$
Communication	Allreduce+Bcast per attempt	one final Gather/Gatherv
CGBN width	per-cofactor (narrowest fit)	once, from bitlen(N)
Signed CGBN	no	yes (`cgbn_signed_rem`)
Fallback	warning + record unfactored	GMP-ECM residual

Both are honest demonstrations of GPU big-integer parallelism, not faster-than-CPU factorizers. Pollard's rho gains only $\sim\sqrt{k}$ from k parallel walks; MPI distribution is a throughput win only with **one rank per distinct GPU** (a multi-GPU host or a cluster) — oversubscribing a single physical GPU with several ranks is measurably *slower*, since the ranks time-slice the one device. For real speed on hard factors the CPU GMP-ECM path (`primefactors.c`) remains the right tool; the value of these two programs is the CGBN parallelism itself and the distributed orchestration around it.